

Практическая работа 1

Чистая машина → инструменты → первый React-проект → красивый интерфейс

Результат занятия: с нуля установленное окружение, запуск React-приложения и сборка первой адаптивной страницы TaskFlow из компонентов.

Как работать с этим методическим указанием.

Методическое указание рассчитано на обучающегося, который раньше не создавал React-проект. Не перескакивайте через шаги. После каждой команды сверяйтесь с блоком «Что должно получиться». Если результат отличается, сначала исправьте текущий шаг и только потом двигайтесь дальше.

1. Все команды вводятся без символа `>` или `$`. В методическом указании они показаны отдельными серыми блоками.
2. Если команда запустила клиент и терминал «завис» – это нормально. Клиент работает. Для остановки используйте `Ctrl+C`.
3. Не копируйте `node_modules` между компьютерами. Зависимости восстанавливаются командой `npm install` в терминал.
4. В первой работе сознательно не подключаем базу данных и сервер. Сначала нужно понять, как устроен интерфейс.

Эта работа должна быть проведена на действительно чистой Windows-машине. Любая команда, требующая скрытой предварительной настройки, считается ошибкой методического указания.

Что получится к концу курса

Этап	Что изучаем	Что появляется в TaskFlow
1	Окружение, React, JSX, компоненты, CSS	Красивый статический дашборд

2	State, события, формы, фильтры, localStorage	Создание и изменение задач в браузере
3	Node.js, Express, HTTP, REST	Собственный API
4	PostgreSQL и SQL	Постоянное хранение задач
5	React Router, fetch, интеграция	Настоящее full-stack приложение
6	Регистрация, вход, сессии, доступ	Личный кабинет и свои задачи
7	Тесты, сборка, адаптивность, публикация	Итоговая версия проекта

Что является итоговой версией

TaskFlow – учебный менеджер задач. В финале пользователь сможет зарегистрироваться, войти, создавать задачи, менять их статус, фильтровать их, открывать отдельные страницы и хранить данные в PostgreSQL. Но цель первой практики: понять путь от пустого компьютера до работающей React-страницы (SPA).

Простая модель: браузер показывает интерфейс. React помогает строить этот интерфейс из небольших компонентов. Позже React будет отправлять запросы Node.js-серверу, а сервер будет работать с PostgreSQL.

Установка инструментов на чистую машину

Устанавливаем Visual Studio Code

VS Code – редактор, в котором будет создаваться код. Для обычной учебной машины используйте User Installer.

Официальная страница: <https://code.visualstudio.com/>

1. Скачайте версию для Windows.
2. Запустите установщик.
3. Оставьте стандартные параметры. Если установщик предлагает добавить команду Code в PATH/контекстное меню – включите эти пункты.
4. После установки закройте старые окна терминала и откройте новое.

Что должно получиться: VS Code запускается из меню Пуск. Позже командой `code` сможем открыть текущую папку прямо из терминала.

Устанавливаем Node.js

Node.js нужен по двум причинам: сначала он запускает инструменты React/Vite, а с третьей практической работы станет основой нашего клиентского приложения. Для курса используем Node.js 24 LTS.

Официальная страница: <https://nodejs.org/>

1. Скачайте LTS-версию Node.js 24 для Windows.
2. Запустите установщик и оставьте стандартные настройки.
3. После установки полностью закройте терминал и откройте новый.

Устанавливаем Git

Git хранит историю проекта. В первой работе используем только три-четыре команды, без веток и сложной теории.

Официальная страница: <https://git-scm.com/>

4. Скачайте Git for Windows.
5. В установщике можно оставить рекомендуемые настройки.
6. После установки откройте новый терминал.

Проверяем, что установка действительно закончена

Откройте PowerShell: Win → введите PowerShell → Enter. Выполните команды по одной:

```
node -v  
npm -v  
git --version  
code --version
```

Что должно получиться: каждая команда печатает номер версии. Для node ожидается строка, начинающаяся с v24. Если вместо версии написано «не является внутренней или внешней командой»/«не распознано», инструмент не попал в PATH или терминал был открыт до установки.

Если PowerShell запрещает npm.ps1: не меняйте политику безопасности вслепую. Для занятия можно открыть Command Prompt (cmd) и выполнять npm-команды там.

Терминал

Терминал – это ещё один способ управлять компьютером. Вместо кликов по папкам мы пишем команды. Важно всегда понимать, в какой папке вы находитесь.

Команда	Что делает	Пример
dir	показывает содержимое текущей папки	dir
cd ПАПКА	переходит в папку	cd Desktop
cd ..	поднимается на уровень выше	cd ..
mkdir ИМЯ	создаёт папку	mkdir taskflow

Правило курса: перед npm install и npm run dev всегда смотрите на путь слева в терминале. Большинство ошибок новичка – команда запущена не в той папке.

Создаём рабочую папку

Для примера создадим проект в Documents. Если вы предпочитаете Desktop, просто перейдите туда.

```
cd $HOME\Documents
mkdir taskflow
cd taskflow
```

Проверка: команда dir должна показывать пустую папку taskflow.

Создаём React-приложение через Vite

Vite создаст правильную структуру React-проекта. Выполните команду:

```
npm create vite@latest client -- --template react
```

После её выполнения появится папка client. Теперь зайдите в неё и установите зависимости:

```
cd client  
npm install
```

Установка может вывести сообщения о количестве пакетов. Главное – чтобы команда завершилась без красной ошибки.

Что произошло: Vite создал исходники, а npm install прочитал package.json и скачал библиотеки проекта в папку node_modules.

Первый запуск

```
npm run dev
```

Терминал напечатает локальный адрес, обычно <http://localhost:5173/>. Нажмите Ctrl и щёлкните по ссылке или скопируйте её в браузер.

Что должно получиться: в браузере открывается стандартная стартовая страница Vite + React. Терминал при этом остаётся занят работающим клиентом разработки – это нормально.

Проверьте «живое обновление»: оставьте браузер открытым. Далее мы изменим файл, и страница обновится почти мгновенно.

Открываем проект в VS Code

Откройте второй терминал или остановите сервер Ctrl+C. Находясь в папке client, выполните:

```
code .
```

Если code не найден: откройте VS Code вручную → File → Open Folder → выберите taskflow\client.

Слева находится Explorer – дерево файлов проекта. Пока запомните только несколько элементов:

Файл/папка	Зачем нужен
src/	наш основной исходный код
src/main.jsx	точка входа React
src/App.jsx	главный компонент страницы
index.html	HTML-оболочка с div id="root"
package.json	список зависимостей и команд npm
node_modules/	установленные библиотеки; вручную не редактируем

Самая важная идея React: страница состоит из компонентов

Обычный HTML можно написать одним огромным файлом. React предлагает разбить интерфейс на понятные части. Например, TaskFlow будет состоять из Sidebar, Header, StatCard и TaskCard. Каждый компонент – обычная JavaScript-функция, которая возвращает JSX-разметку.

JSX простыми словами: это синтаксис, похожий на HTML, который пишется внутри JavaScript. В React используется className вместо class, а JavaScript-значения вставляются в фигурных скобках: {task.title}.

Очищаем шаблон Vite

В папке src удалите ненужные демонстрационные файлы App.css и index.css, а также папку assets, если она содержит только логотип React. Мы создадим свои файлы.

Внутри src создайте две папки: components и data. В итоге структура должна стать такой:

```
src/
```

```
components/  
data/  
App.jsx  
main.jsx  
styles.css
```

Как создать папку в VS Code: в Explorer наведите курсор на src → кнопка New Folder. Файл создаётся кнопкой New File.

Подключаем главный файл приложения

Полностью замените содержимое src/main.jsx:

```
import { StrictMode } from 'react';  
import { createRoot } from 'react-dom/client';  
import App from './App.jsx';  
import './styles.css';  
  
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <App />  
  </StrictMode>,  
);
```

Разберём код: createRoot находит <div id="root"> в index.html и помещает туда наш компонент App. Строка import './styles.css' подключает общие стили.

Создаём демонстрационные данные

Создайте src/data/tasks.js. Пока задачи будут обычным JavaScript-массивом. Позже тот же интерфейс начнёт получать данные с сервера.

```
export const tasks = [  
  {
```

```
id: 1,
title: 'Подготовить макет главной страницы',
project: 'Учебный проект',
status: 'В работе',
priority: 'Высокий',
dueDate: '20 сентября',
},
{
  id: 2,
  title: 'Разобраться с компонентами React',
  project: 'Обучение',
  status: 'Нужно сделать',
  priority: 'Средний',
  dueDate: '22 сентября',
},
{
  id: 3,
  title: 'Оформить README проекта',
  project: 'Учебный проект',
  status: 'Готово',
  priority: 'Низкий',
  dueDate: '18 сентября',
},
{
  id: 4,
  title: 'Проверить мобильную версию',
  project: 'Интерфейс',
  status: 'Нужно сделать',
```

```
priority: 'Средний',  
dueDate: '24 сентября',  
},  
];
```

Почему id уже нужен: React должен отличать одну карточку от другой. В первой работе достаточно простых числовых id. Позже сервер и база данных будут выдавать настоящие идентификаторы.

Боковое меню

Компонент отвечает только за левую часть интерфейса. Создайте файл `./components/Sidebar.jsx`

```
export default function Sidebar() {  
  return (  
    <aside className="sidebar">  
      <div className="brand">  
        <div className="brandMark">T</div>  
        <div>  
          <strong>TaskFlow</strong>  
          <span>student edition</span>  
        </div>  
      </div>  
  
      <nav className="menu" aria-label="Основная навигация">  
        <button className="menuItem active" type="button">Обзор</button>  
        <button className="menuItem" type="button">Мои задачи</button>  
        <button className="menuItem" type="button">Доска</button>  
        <button className="menuItem" type="button">Проекты</button>  
      </nav>  
    </div>  
  );  
}
```

```
<div className="sidebarNote">
  <strong>Практика 1</strong>
  <span>Статический интерфейс на React</span>
</div>
</aside>
);
}
```

Верхняя часть страницы

Здесь находится заголовок и кнопка новой задачи. Пока кнопка только выглядит как настоящая — действие добавим в практике 2. Создайте файл `./components/Header.jsx`

```
export default function Header() {
  return (
    <header className="topbar">
      <div>
        <p className="eyebrow">Учебный проект</p>
        <h1>Добро пожаловать в TaskFlow</h1>
        <p className="subtitle">Здесь будут собраны ваши задачи, проекты и
прогресс.</p>
      </div>

      <button className="primaryButton" type="button">
        + Новая задача
      </button>
    </header>
  );
}
```

```
}
```

Карточка статистики

Этот компонент уже получает данные через props. Один и тот же шаблон мы используем четыре раза с разными значениями. Создайте файл `./components/StatCard.jsx`

```
export default function StatCard({ label, value, note }) {  
  return (  
    <article className="statCard">  
      <span>{label}</span>  
      <strong>{value}</strong>  
      <small>{note}</small>  
    </article>  
  );  
}
```

Карточка задачи

Карточка получает объект `task` и показывает его поля. Функция `getStatusClass` выбирает CSS-класс в зависимости от статуса. Создайте файл `./components/TaskCard.jsx`

```
function getStatusClass(status) {  
  if (status === 'Готово') return 'status done';  
  if (status === 'В работе') return 'status progress';  
  return 'status todo';  
}  
  
export default function TaskCard({ task }) {  
  return (  
    <article className="taskCard">
```

```

<div className="taskMain">
  <div className="taskTopLine">
    <span className={getStatusClass(task.status)}>{task.status}</span>
    <span className="priority">{task.priority}</span>
  </div>

  <h3>{task.title}</h3>
  <p>{task.project}</p>
</div>

<div className="taskDate">
  <span>Срок</span>
  <strong>{task.dueDate}</strong>
</div>
</article>
);
}

```

11. Собираем страницу в App.jsx

Теперь главный компонент не содержит весь сайт целиком. Он соединяет готовые детали.

```

import Header from './components/Header.jsx';
import Sidebar from './components/Sidebar.jsx';
import StatCard from './components/StatCard.jsx';
import TaskCard from './components/TaskCard.jsx';
import { tasks } from './data/tasks.js';

export default function App() {

```

```
return (  
  <div className="appShell">  
    <Sidebar />  
  
    <main className="content">  
      <Header />  
  
      <section className="statsGrid" aria-label="Статистика">  
        <StatCard label="Всего задач" value="12" note="+3 за эту неделю" />  
        <StatCard label="В работе" value="4" note="Нужны сегодня" />  
        <StatCard label="Готово" value="6" note="Хороший темп" />  
        <StatCard label="Проектов" value="3" note="Учёба и портфолио" />  
      </section>  
  
      <section className="panel">  
        <div className="sectionHeading">  
          <div>  
            <p className="eyebrow">Фокус</p>  
            <h2>Ближайшие задачи</h2>  
          </div>  
          <button className="ghostButton" type="button">Показать все</button>  
        </div>  
  
        <div className="taskList">  
          {tasks.map((task) => (  
            <TaskCard key={task.id} task={task} />  
          ))}  
        </div>  
      </section>  
    </main>  
  </div>  
)
```

```
    </section>
  </main>
</div>
);
}
```

Обратите внимание: `tasks.map(...)` проходит по массиву задач и для каждой записи создаёт `TaskCard`. `key={task.id}` помогает React отличать элементы списка.

Делаем сайт похожим на продукт

Создайте `src/styles.css` и вставьте стили ниже. Не пытайтесь запомнить каждое свойство. На этом занятии важно увидеть, как классы из JSX связываются с CSS.

```
* {
  box-sizing: border-box;
}

:root {
  font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont,
  "Segoe UI", sans-serif;
  color: #172033;
  background: #f5f7fb;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
}

body {
  margin: 0;
```

```
min-width: 320px;  
min-height: 100vh;  
background: #f5f7fb;  
}
```

```
button {  
  font: inherit;  
}
```

```
button:focus-visible {  
  outline: 3px solid rgba(79, 70, 229, 0.28);  
  outline-offset: 2px;  
}
```

```
.appShell {  
  min-height: 100vh;  
  display: grid;  
  grid-template-columns: 260px 1fr;  
}
```

```
.sidebar {  
  min-height: 100vh;  
  padding: 28px 20px;  
  background: #111827;  
  color: white;  
  display: flex;  
  flex-direction: column;  
  gap: 34px;
```

```
}
```

```
.brand {  
  display: flex;  
  align-items: center;  
  gap: 12px;  
}
```

```
.brandMark {  
  width: 42px;  
  height: 42px;  
  border-radius: 13px;  
  display: grid;  
  place-items: center;  
  background: #ffffff;  
  color: #111827;  
  font-weight: 800;  
  font-size: 20px;  
}
```

```
.brand strong,  
.brand span {  
  display: block;  
}
```

```
.brand span {  
  margin-top: 2px;  
  color: #9ca3af;
```

```
font-size: 12px;  
}
```

```
.menu {  
  display: grid;  
  gap: 8px;  
}
```

```
.menuItem {  
  width: 100%;  
  border: 0;  
  border-radius: 12px;  
  padding: 12px 14px;  
  text-align: left;  
  color: #cbd5e1;  
  background: transparent;  
  cursor: pointer;  
}
```

```
.menuItem:hover,  
.menuItem.active {  
  color: white;  
  background: #1f2937;  
}
```

```
.sidebarNote {  
  margin-top: auto;  
  padding: 15px;
```

```
border: 1px solid #374151;
border-radius: 14px;
background: #182131;
}
```

```
.sidebarNote strong,
.sidebarNote span {
  display: block;
}
```

```
.sidebarNote span {
  margin-top: 5px;
  color: #9ca3af;
  font-size: 12px;
  line-height: 1.5;
}
```

```
.content {
  width: min(1180px, 100%);
  margin: 0 auto;
  padding: 42px;
}
```

```
.topbar {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 24px;
```

```
margin-bottom: 28px;  
}
```

```
.eyebrow {  
  margin: 0 0 6px;  
  color: #4f46e5;  
  font-weight: 800;  
  font-size: 12px;  
  letter-spacing: 0.08em;  
  text-transform: uppercase;  
}
```

```
h1,  
h2,  
h3,  
p {  
  margin-top: 0;  
}
```

```
h1 {  
  margin-bottom: 8px;  
  font-size: clamp(28px, 4vw, 42px);  
  letter-spacing: -0.03em;  
}
```

```
.subtitle {  
  margin-bottom: 0;  
  color: #667085;
```

```
    line-height: 1.6;
}
```

```
.primaryButton,  
.ghostButton {  
    border-radius: 12px;  
    padding: 11px 16px;  
    cursor: pointer;  
    font-weight: 700;  
}
```

```
.primaryButton {  
    border: 1px solid #4f46e5;  
    background: #4f46e5;  
    color: white;  
}
```

```
.primaryButton:hover {  
    background: #4338ca;  
}
```

```
.ghostButton {  
    border: 1px solid #d9deea;  
    background: white;  
    color: #344054;  
}
```

```
.statsGrid {
```

```
display: grid;
grid-template-columns: repeat(4, minmax(0, 1fr));
gap: 16px;
margin-bottom: 22px;
}
```

```
.statCard,
.panel {
border: 1px solid #e4e7ec;
background: white;
box-shadow: 0 12px 32px rgba(16, 24, 40, 0.04);
}
```

```
.statCard {
padding: 20px;
border-radius: 16px;
}
```

```
.statCard span,
.statCard small {
display: block;
color: #667085;
}
```

```
.statCard strong {
display: block;
margin: 8px 0 4px;
font-size: 32px;
```

```
}
```

```
.panel {  
  padding: 22px;  
  border-radius: 18px;  
}
```

```
.sectionHeading {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  gap: 16px;  
  margin-bottom: 16px;  
}
```

```
.sectionHeading h2 {  
  margin-bottom: 0;  
}
```

```
.taskList {  
  display: grid;  
  gap: 10px;  
}
```

```
.taskCard {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;
```

```
gap: 20px;
padding: 17px;
border: 1px solid #eaecf0;
border-radius: 14px;
background: #fcfcfd;
}
```

```
.taskTopLine {
  display: flex;
  align-items: center;
  flex-wrap: wrap;
  gap: 8px;
  margin-bottom: 9px;
}
```

```
.status,
.priority {
  display: inline-flex;
  align-items: center;
  border-radius: 999px;
  padding: 5px 9px;
  font-size: 12px;
  font-weight: 800;
}
```

```
.status.todo {
  color: #344054;
  background: #f2f4f7;
```

```
}
```

```
.status.progress {  
  color: #3730a3;  
  background: #eef2ff;  
}
```

```
.status.done {  
  color: #067647;  
  background: #ecfdf3;  
}
```

```
.priority {  
  color: #b54708;  
  background: #fffaeb;  
}
```

```
.taskCard h3 {  
  margin-bottom: 5px;  
  font-size: 16px;  
}
```

```
.taskCard p {  
  margin-bottom: 0;  
  color: #667085;  
  font-size: 14px;  
}
```

```
.taskDate {  
  min-width: 115px;  
  text-align: right;  
}
```

```
.taskDate span,  
.taskDate strong {  
  display: block;  
}
```

```
.taskDate span {  
  margin-bottom: 4px;  
  color: #98a2b3;  
  font-size: 12px;  
}
```

```
.taskDate strong {  
  font-size: 13px;  
}
```

```
@media (max-width: 900px) {  
  .appShell {  
    grid-template-columns: 1fr;  
  }  
}
```

```
.sidebar {  
  min-height: auto;  
  padding: 18px;
```

```
}
```

```
.menu {  
  grid-template-columns: repeat(4, minmax(0, 1fr));  
}
```

```
.sidebarNote {  
  display: none;  
}
```

```
.statsGrid {  
  grid-template-columns: repeat(2, minmax(0, 1fr));  
}  
}
```

```
@media (max-width: 640px) {  
  .content {  
    padding: 24px 16px;  
  }  
}
```

```
.topbar,  
.sectionHeading,  
.taskCard {  
  align-items: stretch;  
  flex-direction: column;  
}
```

```
.primaryButton,
```

```
.ghostButton {  
  width: 100%;  
}  
  
.statsGrid,  
.menu {  
  grid-template-columns: 1fr 1fr;  
}  
  
.taskDate {  
  text-align: left;  
}  
}
```

13. Настраиваем index.html

Откройте index.html в корне client и замените содержимое:

```
<!doctype html>  
<html lang="ru">  
  <head>  
    <meta charset="UTF-8" />  
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />  
    <meta name="description" content="TaskFlow – учебный менеджер задач на  
React и Node.js" />  
    <title>TaskFlow — мои задачи</title>  
  </head>  
  <body>  
    <div id="root"></div>  
    <script type="module" src="/src/main.jsx"></script>
```

```
</body>  
</html>
```

Зачем lang="ru": браузер и вспомогательные технологии понимают основной язык страницы. meta viewport нужен для корректной мобильной ширины.

Запускаем итог практики

Если сервер разработки был остановлен, откройте терминал VS Code (Terminal → New Terminal) и убедитесь, что терминал находится в папке client. Затем:

```
npm run dev
```

Откройте адрес из терминала. Вы должны увидеть тёмное боковое меню, заголовок TaskFlow, четыре карточки статистики и список задач.

Главная проверка: уменьшите ширину окна браузера. При ширине около планшета боковое меню должно перейти наверх, а на телефоне карточки должны перестроиться без горизонтальной прокрутки.

Первый эксперимент – обязательно сделать руками

Выполните три изменения без подсказки кода:

1. В tasks.js измените название одной задачи на свою.
2. Добавьте пятую задачу, скопировав структуру существующей. У неё должен быть новый id.
3. В Header.jsx измените подзаголовок страницы.

Что вы тренируете: поиск нужного файла, изменение массива объектов, понимание того, что данные и отображение можно хранить отдельно.

16. Сохраняем проект в Git

Остановите сервер Ctrl+C или откройте второй терминал. Находясь в client, перейдите в корень taskflow и создайте репозиторий:

```
cd ..  
git init  
git add .  
git status
```

Если Git впервые просит имя и email, настройте их один раз на компьютере:

```
git config --global user.name "Имя Фамилия"  
git config --global user.email "student@example.com"
```

Теперь создайте первый коммит:

```
git commit -m "Практика 1: первый интерфейс TaskFlow"
```

Проверка: git status должен сообщить, что рабочая директория чистая. Папка node_modules не должна попадать в коммит — Vite создаёт .gitignore.

Если что-то не работает

Проблема	Почему бывает	Что сделать
node/npm не распознано	терминал открыт до установки или Node не в PATH	закрыть все терминалы; открыть новый; при необходимости переустановить Node LTS
npm.ps1 cannot be loaded	PowerShell блокирует скрипт npm.ps1	для занятия открыть cmd; не отключать защиту случайной командой из интернета
npm run dev: package.json not found	команда запущена не в client	выполнить dir и cd client
порт 5173 занят	уже работает другой Vite	использовать адрес, который предложит Vite, либо остановить старый процесс Ctrl+C

белая страница	ошибка JavaScript/импорта	посмотреть красную ошибку в терминале и DevTools → Console; проверить имена файлов и регистр букв
Failed to resolve import	файл лежит не там или неверно написан путь	сверить src/components и строки import
стили не применились	styles.css не импортирован	проверить import './styles.css' в main.jsx
после изменения ничего не меняется	редактируется другой проект или сервер остановлен	проверить путь папки VS Code и адрес из npm run dev

Что студент должен уметь объяснить после работы

1. Чем VS Code отличается от Node.js?
2. Зачем нужен npm?
3. Что делает команда npm run dev?
4. В чём разница между App.jsx и main.jsx?
5. Что такое React-компонент?
6. Что такое props на примере StatCard?
7. Почему для списка задач используется map?
8. Зачем элементу списка key?
9. Где находятся данные задач и где находится их отображение?
10. Почему node_modules не нужно передавать преподавателю?

Что сдавать

1. Папку проекта или ссылку на Git-репозиторий без node_modules.
2. Скриншот TaskFlow на обычной ширине.
3. Скриншот мобильной версии из DevTools или узкого окна браузера.
4. Не менее пяти собственных задач в tasks.js.
5. Первый Git-коммит.

Самостоятельная работа

Сделайте TaskFlow немного своим, но не ломайте структуру:

4. Добавьте шестую задачу.
5. Измените текст блока «Практика 1» в боковом меню на своё имя/группу.
6. Добавьте в статистику пятую карточку «Просрочено» и добейтесь аккуратного переноса сетки.
7. Измените один акцентный цвет сайта, но сохраните читаемость текста.
8. Подготовьте короткий ответ: «Как данные из tasks.js попадают на экран?» — 4–6 предложений.

Что будет в практике 2

Сейчас TaskFlow красивый, но «неживой»: кнопки ничего не меняют, а после перезагрузки новые задачи исчезли бы. На следующем занятии разберём useState, события, формы и localStorage. После практики 2 пользователь уже сможет создавать, редактировать, удалять и фильтровать задачи прямо в браузере.

Официальные материалы

Node.js — загрузка и LTS: <https://nodejs.org/>

Vite — Getting Started: <https://vite.dev/guide/>

React — документация: <https://react.dev/learn>

VS Code — установка на Windows:

<https://code.visualstudio.com/docs/setup/windows>

Git — официальный сайт: <https://git-scm.com/>